



ENTREGA 2: CASO 17 CESFAM

Repositorio: <https://github.com/diegovilloutafredes/caso-17-cesfam-lenguajes-de-programacion-www>

Integrantes:

Bastian Camus 202173013-3
Gaspar Navarro 202173003-6
Diego Villouta 2773019-1
Carolina Sire 202173105-9
Badir Villegas 202273020-K
Benjamin Paulsen 202173017-6

ÍNDICE

Contexto del problema	1
3.1. 1. Diagrama de clases con los microservicios (15 pts)	2
3.2. 2. Relaciones, composición y herencia (10 pts)	5
3.3. 3. Separación de responsabilidades (15 pts)	6
3.4. 4. Estructura de microservicio (10 pts)	10
3.5. 5. Alta cohesión de microservicios (10 pts)	12
3.6. 6. Bajo acoplamiento de microservicios (15 pts)	13
4. Formato esperado para justificar decisiones	15
Diagrama de Arquitectura de Infraestructura y Datos.	15
ANEXOS Y REFERENCIAS	16
Diagrama de Clases	16

Contexto del problema

Sistema: automatización de la libreta de medicamentos del CESFAM. El sistema gestiona pacientes, ficha clínica, recetas emitidas por médicos, reservas y entregas de medicamentos en farmacia, control de inventario por lotes, (batches), bajas de stock (write-offs), notificaciones a pacientes, y reportes operacionales.

Stack: Python 3.12, FastAPI, Pydantic, Docker Compose. Almacenamiento *in-memory* (sandbox; el proyecto real usaría PostgreSQL).

Artefactos entregados:

- Caso17_Diagrama_Clases.drawio: modelo de dominio (14 entidades agrupadas por color de bounded context).
- Caso17_Diagrama_Microservicios.drawio: vista de contratos inter-servicio.
- Caso17_Diagrama_Arquitectura_de_Infraestructura_y_Datos.drawio : Arquitectura e Infraestructura de datos
- backend-microservices/: 7 servicios (identity, patient, inventory, prescription, notification, report, api_gateway) + shared/ con utilidades transversales.

3.1.1. Diagrama de clases con los microservicios (15 pts)

Primero, sobre el concepto de agregado: es un grupo de entidades del dominio que se modifican juntas y comparten una regla de consistencia, (un "invariante transaccional", es decir, una condición matemática que debe ser verdadera antes y después de cada transacción, sin importar los cambios intermedios). Por ejemplo, en el agregado **Inventory** el invariante exige que tres cantidades sean siempre iguales: **stock.physical**, (el total de unidades en bodega), **stock.available** + **stock.reserved**, (lo libre para nuevas reservas más lo retenido por recetas pendientes de retiro), y $\Sigma(\text{batch.available})$, (la suma de unidades disponibles en cada lote físico). La igualdad se expresa como **stock.physical = stock.available + stock.reserved = $\Sigma(\text{batch.available})$** . Toda operación sobre el agregado debe ser atómica, (se aplica completa o no se aplica). Por eso un agregado nunca debe dividirse entre dos microservicios; separarlo obliga a coordinación distribuida para mantener la invariante.

Decisión arquitectónica 1: Cómo descomponer el sistema CESFAM en microservicios

Decisión arquitectónica: cómo descomponer el sistema CESFAM en microservicios y agrupar las 14 entidades del dominio del diagrama de clases, (User, Patient, Guardian, PatientCard, MedicationLog, Prescription, PrescriptionItem, Medication, Stock, Batch, MedicationDelivery, DeliveryBatch, WriteOff, Notification, Report).

Alternativa A: Monolito modular.

Un único proceso FastAPI con módulos internos (patients/, prescriptions/, inventory/, identity/, notifications/, reports/) y una base de datos compartida.

Descarte de A:

Un monolito acopla despliegue, (cualquier cambio mínimo implica redespregar todo el sistema), impide escalar contextos asimétricamente, (Inventory recibe carga distinta a Notification), y obliga a coordinar releases entre todos los dominios.

Alternativa B: CRUD-by-entity, un servicio por entidad.

Cada clase del diagrama se convierte en su propio servicio: PatientService, GuardianService, PatientCardService, MedicationLogService, PrescriptionService, PrescriptionItemService, MedicationService, StockService, BatchService, MedicationDeliveryService, DeliveryBatchService, WriteOffService, NotificationService, ReportService, IdentityService. Dando un total de 15 microservicios.

Descarte de B:

Rompe agregados con invariantes transaccionales. Separar Stock y Batch de Medication exige coordinar tres servicios cada vez que se reserva o consume stock. Lo mismo aplica a Prescription y PrescriptionItem, un ítem sin su receta asociada, no tiene sentido de existir, y crear una receta sería una transacción distribuida. Además que operacionalmente, se convierte en mantener 15 servicios, y todo lo que eso conlleva, (15 deploys, 15 health checks, 15 logs, 15 redes Docker), sin ganar cohesión funcional.

Alternativa C: Por capa técnica.

Tres microservicios separando responsabilidades técnicas en lugar de funcionales:

- ApiService: todos los endpoints REST de todos los dominios.
- BusinessService: lógica de negocio entre dominios.
- DataService: persistencia.

Descarte de C:

Acopla el sistema en el eje equivocado, (capa técnica) en lugar del eje natural, (dominio). Cualquier cambio del dominio, (por ejemplo, agregar un nuevo estado PrescriptionStatus.SHIPPED_TO_HOME), obliga a tocar los tres servicios. La cohesión funcional se rompe, la lógica que pertenece junta, (todo lo de receta), queda dispersa entre los tres. Es un anti patrón en arquitectura de microservicios.

Alternativa D: Un microservicio por agregado.

Granularidad intermedia con 9 microservicios donde cada agregado se vuelve un servicio:

- PatientService (Patient, Guardian)
- PatientHistoryService (PatientCard, MedicationLog)
- MedicationCatalogService (Medication)
- StockService (Stock, Batch, WriteOff)
- PrescriptionIssuingService (Prescription, PrescriptionItem)
- PrescriptionDispensingService (MedicationDelivery, DeliveryBatch)
- NotificationService
- ReportService

- IdentityService

Descarte de D:

Sigue dividiendo agregados con invariantes locales, aunque menos que la Alternativa B. Persisten dos problemas. Primero: MedicationCatalogService y StockService separados requieren coordinar reservas. Segundo: PrescriptionIssuingService y PrescriptionDispensingService pueden tener inconsistencias si el estado de una receta cambia, (una receta cancelada en Issuing que Dispensing todavía cree READY). Además las llamadas inter-servicio aumentan, el frontend para mostrar el detalle de una receta debe consultar 4 servicios, contra 2 con bounded contexts más amplios.

Alternativa E: Híbrido (dominios de negocio + funciones transversales).

Distingue dos categorías de servicios:

1. Servicios de dominio, (bounded contexts del negocio CESFAM): Patient, Inventory, Prescription. Reflejan el lenguaje del usuario final, (médico y personal de farmacia).
2. Servicios funcionales transversales, (utilizados por todos los dominios): Identity, Notification, Report. No son dominio del negocio, sino capacidades que sistema requiere.

Más un ApiGateway BFF. Llegando a un total de 6 servicios de dominio con un 1 BFF, siendo así 7 procesos.

Justificación de elección:

La Alternativa E cumple los cuatro criterios aplicados al diagrama de clases:

1. Invariantes transaccionales locales: Las entidades cuyo invariante exige atomicidad se mantienen en el mismo servicio. Medication + Stock + Batch mantienen la coherencia entre las tres vistas del stock, (total, disponible más reservado, suma por lote). Prescription + PrescriptionItem + MedicationDelivery + DeliveryBatch mantienen el ciclo de vida atómico de la receta. Separar entidades que comparten un invariante implicaría transacciones distribuidas.
2. Frecuencia de cambio: Entidades que mutan juntas viven juntas. Patient + Guardian + PatientCard + MedicationLog mutan juntos cuando se registra o actualiza un paciente. Las migraciones de schema afectan a las cuatro entidades simultáneamente. Separarlas aumenta los pasos operativos de cada despliegue.
3. Lenguaje ubicuo: Cada servicio "habla un único idioma" interno. PatientService habla de personas, parentesco, ficha clínica e historial. InventoryService habla de catálogo, cantidades, lotes y mermas. PrescriptionService habla de prescripción, dosis, entrega y despacho. Mezclar lenguajes diluye la cohesión semántica.
4. Independencia de despliegue: Los 6 servicios de dominio son desplegables de manera independiente. Cambiar el schema interno de PrescriptionService no requiere tocar PatientService ni InventoryService, sólo si cambia el contrato HTTP entre ellos.

El resultado se materializa en el diagrama de clases, (entidades agrupadas por color).

3.2.2. Relaciones, composición y herencia (10 pts)

Decisión arquitectónica 1: Modelado de Roles de Usuarios (Médico y Farmacia)

Decisión arquitectónica: Representación de los usuarios (Doctor y PharmacyStaff) en el sistema.

Alternativa A (Descartada): Herencia con subclases a partir de una clase base User.

Razón del descarte: Se evaluó inicialmente, pero se descartó porque ambos roles comparten exactamente los mismos atributos demográficos (id, rut, fullName). La separación por subclases no aportaba polimorfismo real en la base de datos PostgreSQL, y habría forzado un mapeo relacional complejo para diferencias que son solo permisos a nivel de servicios (capa de aplicación).

Alternativa B (Seleccionada): Asociación dirigida con Atributo de Control (Enum). Una única clase User con el atributo role: UserRole (DOCTOR o PHARMACY_STAFF).

Justificación de la decisión: Se elige la **Alternativa B** porque simplifica la persistencia en una sola tabla en PostgreSQL y dota al sistema de alta flexibilidad. Si en el futuro se incorporan nuevos roles, solo se extiende el enumerado, evitando migraciones estructurales o refactorizaciones de clases.

Decisión arquitectónica 2: Modelado de Inventario (Medication, Stock y Batch)

Decisión arquitectónica: Tipo de relación para el catálogo de medicamentos, sus existencias lógicas y sus lotes físicos.

Alternativa A (Descartada): Agregación (◇) (Relación débil).

Razón del descarte: Se descarta porque en el CESFAM un lote (Batch) o las existencias (Stock) no tienen identidad propia ni sentido operacional sin el medicamento al que pertenecen. Permitir agregación daría pie a "lotes huérfanos" en el sistema.

Alternativa B (Seleccionada): Composición Fuertemente Acoplada (◆) (Medication ◆— Stock y Medication ◆— Batch).

Justificación de la decisión: Se elige la **Alternativa B** porque la existencia de las partes depende estrictamente del todo. Si un medicamento se elimina de manera

definitiva, sus lotes y stock deben borrarse o inactivarse en cascada, asegurando la consistencia interna en InventoryService.

Decisión arquitectónica 3: Modelado de Estructura de Recetas (Prescription y PrescriptionLine)

Decisión arquitectónica: Tipo de relación entre la cabecera de la receta y el desglose de sus fármacos.

Alternativa A (Descartada): Asociación Simple Dirigida (Entidades independientes).

Razón del descarte: Se descarta porque una línea de medicamento detallada (dosis, cantidad) carece de semántica aislada; no puede existir ni procesarse si no pertenece a un folio de receta matriz legítimo.

Alternativa B (Seleccionada): Composición (◆)
(Prescription ◆— PrescriptionLine).

Justificación de la decisión: Se elige la **Alternativa B** debido a la dependencia absoluta del ciclo de vida. Las líneas nacen y mueren de forma conjunta con la receta principal, garantizando la integridad transaccional del agregado.

Decisión arquitectónica 4: Referencias entre Contextos (PrescriptionLine hacia Medication)

Decisión arquitectónica: Conexión entre contextos delimitados distintos (recetas en PrescriptionService e insumos en InventoryService).

Alternativa A (Descartada): Asociación Bidireccional o Embebido de Objetos.

Razón del descarte: Violaría los fundamentos de los microservicios. Si InventoryService modificara la estructura o atributos de Medication, rompería inmediatamente la persistencia y validación del servicio de recetas por acoplamiento de datos.

Alternativa B (Seleccionada): Asociación Unidireccional Débil (Referencia por ID) mediante el atributo medicationId: Integer.

Justificación de la decisión: Se elige la **Alternativa B**. Al guardar sólo un identificador numérico, se respetan las fronteras de red. Los datos del fármaco se consultan On-Demand vía API Gateway cuando el flujo lo requiere, manteniendo las bases de datos de cada servicio completamente independientes.

3.3. 3. Separación de responsabilidades (15 pts)

1. Microservicio: PatientService (Servicio de Pacientes)

Decisión arquitectónica:

Delimitación de responsabilidades de información demográfica, tutores y ficha del paciente.

Alternativa A: Incluir el historial de recetas y las entregas físicas de medicamentos dentro del mismo servicio del Paciente para centralizar su ficha médica.

Descarte: Se descarta porque sobrecarga el dominio del paciente con lógica transaccional y operacional de farmacia (fechas de vencimiento, recetas pendientes, control de stock), por lo que no existiría una única responsabilidad.

Alternativa B: Crear un microservicio exclusivo para la gestión de entidades de pacientes, sus datos personales, la asociación con sus tutores y la consulta de su carnet de atención.

Selección: Se selecciona la alternativa B.

Justificación: El dominio de Pacientes cambia con una frecuencia y naturaleza distinta al de las recetas médicas. Aislarlo permite que la información demográfica y legal de los tutores esté centralizada, sirviendo de fuente de verdad para el resto del ecosistema sin mezclarse con flujos logísticos

2. Microservicio: PrescriptionService (Servicio de Prescripciones)

Decisión arquitectónica:

Centralización del ciclo de vida de las recetas médicas y su flujo de despacho.

Alternativa A: Permitir que el servicio de prescripciones maneje directamente el inventario físico de las cajas de medicamentos al momento de guardarse o despacharse.

Descarte: Se descarta porque genera un acoplamiento directo con la bodega física y los lotes específicos de medicamentos, impidiendo que el inventario funcione de manera autónoma para compras o mermas..

Alternativa B: Establecer un servicio enfocado puramente en las recetas , sus líneas de tratamiento , plazos de retiro, estados de transición y el registro de la entrega final consumiendo otros servicios.

Selección: Se selecciona la alternativa B.

Justificación: La receta médica es el núcleo transaccional del negocio del CESFAM. Mantenerla independiente permite encapsular las complejas reglas de sus estados de transición y delegar la validación del paciente o el descuento de existencias a sus respectivos microservicios especialistas.

3. Microservicio: InventoryService (Servicio de Inventario y Stock)

Decisión arquitectónica:

Aislamiento del control físico de existencias, lotes y mermas de medicamentos.

Alternativa A: Manejar el stock de forma global integrado en el módulo de recetas, asumiendo que el stock sólo varía cuando un médico emite o la farmacia entrega una prescripción.

Descarte: Se descarta porque ignora los eventos administrativos críticos de un inventario clínico como el ingreso de nuevos cargamentos, el control estricto de vencimientos por lote, y las mermas por cajas dañadas.

Alternativa B: Crear un microservicio independiente encargado del catálogo de fármacos, el control de cantidades físicas, disponibles y reservadas, el seguimiento de lotes y los procesos de baja de insumos.

Selección: Se selecciona la alternativa B.

Justificación: Permite alertar de stocks críticos o procesar mermas de manera aislada. La separación asegura que la consistencia del stock físico no dependa de los flujos de atención clínica directos de las recetas.

4. Microservicio: NotificationService (Servicio de Notificaciones)

Decisión arquitectónica:

Desacoplamiento de los canales de comunicación y mensajería hacia pacientes y tutores.

Alternativa A: Hacer que el servicio de Prescripciones o de Inventario envíe directamente los correos electrónicos utilizando librerías internas.

Descarte: Se descarta porque añade responsabilidades de infraestructura de red y manejo de fallas de proveedores externos (SMTP/SMS gateways) a dominios clínicos críticos, bloqueando transacciones médicas si el proveedor de mensajería falla.

Alternativa B: Extraer toda la mensajería a un microservicio genérico que reciba órdenes de alerta, determine el canal, gestione el evento y registre su estado de envío.

Selección: Se selecciona la alternativa B.

Justificación: El servicio se vuelve reutilizable por cualquier parte del sistema del CESFAM. Si en el futuro se decide cambiar el proveedor de correo o agregar alertas vía WhatsApp, las modificaciones se limitan exclusivamente a este microservicio sin alterar la lógica de recetas ni inventarios.

5. Microservicio: IdentityService (Servicio de Identidad y Acceso)

Decisión arquitectónica:

Centralización de la autenticación y control de accesos basado en roles clínicos.

Alternativa A: Almacenar las credenciales y perfiles de los usuarios dentro de cada microservicio para que cada uno valide de forma autónoma quién interactúa con sus endpoints.

Descarte: Se descarta debido al alto riesgo de seguridad, redundancia de datos de usuarios y la extrema dificultad para mantener una sesión unificada.

Alternativa B: Establecer un servicio perimetral dedicado a la autenticación y a la validación de tokens , exponiendo la identidad y el rol que rige las operaciones autorizadas del sistema.

Selección: Se selecciona la alternativa B.

Justificación: Mantiene la seguridad como una capa transversal y estandarizada. Los demás microservicios confían en el token validado por este componente y se enfocan estrictamente en ejecutar las reglas de negocio de sus propios dominios sin cuestionar la legitimidad de la sesión del usuario.

6. Microservicio: ReportService (Servicio de Reportes)

Decisión arquitectónica:

Aislamiento de procesos intensivos de análisis y exportación de datos.

Alternativa A: Implementar los métodos de generación de informes pesados (como el consolidado de medicamentos vencidos o reservas mensuales) directamente dentro de InventoryService o PrescriptionService.

Descarte: Se descarta porque las consultas analíticas pesadas que abarcan grandes volúmenes de datos históricos degradan el rendimiento de las bases de datos operacionales, afectando la velocidad de atención a los pacientes en ventanilla.

Alternativa B: Crear un microservicio especializado que consuma datos de manera asíncrona o mediante llamadas optimizadas inter-servicio a través de y con el único fin de procesar, estructurar y exportar archivos.

Selección: Se selecciona la alternativa B.

Justificación: Aísla la carga computacional intensiva de la generación de documentos analíticos de las operaciones transaccionales del día a día del CESFAM, asegurando que la farmacia e inventario mantengan alta disponibilidad en todo momento.

3.4. 4. Estructura de microservicio (10 pts)

Qué estructura interna seleccionó. Qué alternativas de organización descartó. Por qué la estructura facilita mantenimiento y evolución.

Decisión arquitectónica: Se define una estructura modular y en capas para cada microservicio del sistema CESFAM.

Alternativa A: Construir cada microservicio usando una estructura donde los endpoints, lógica de negocio, validaciones y acceso a datos están directamente implementados en los mismos archivos.

Descarte: Se descarta la alternativa A porque es una arquitectura poco mantenible y escalable en el tiempo, muy acoplada, tal que mezclas rutas http, reglas del negocio, validaciones hace que se duplique código y su mantención o cambios se complejiza innecesariamente. Además, cualquier cambio a la BD o lógica impacta en los endpoints del microservicio. También se puede considerar la complejidad a la hora de hacer qa y reutilizar componentes.

Alternativa B: Estructura modular separada por responsabilidades, usando carpetas y componentes especializados, mediante la siguiente separación:

- API Gateway/BFF
- Controladores/API REST
- Lógica de negocio
- Modelo y validaciones
- Persistencia de datos

Selección: Se selecciona la alternativa B.

Justificación: La arquitectura backend del sistema CESFAM fue diseñada utilizando microservicios especializados organizados mediante capas y módulos independientes.

El sistema incorpora un API Gateway/BFF encargado de:

- validación de autenticación,
- enrutamiento de solicitudes,
- agregación de datos.

Esto permite desacoplar a los microservicios de la capa cliente y centralizar responsabilidades comunes.

La capa de negocio se divide en microservicios especializados:

- IdentityService
- PatientService
- InventoryService
- PrescriptionService
- NotificationService
- ReportService

Cada microservicio mantiene una responsabilidad específica del dominio, mejorando cohesión y permitiendo evolución independiente.

Internamente, los microservicios utilizan una estructura modular organizada mediante componentes como:

- routers/controladores,
- schemas/modelos,
- acceso a datos y persistencia.

Esta separación evita mezclar:

- endpoints REST,
- validaciones,
- reglas de negocio,
- consultas a base de datos.

Además, la persistencia se centraliza mediante PostgreSQL utilizando el patrón Shared Database Pattern.

De esta manera, es posible separar claramente las responsabilidades dentro del sistema. Los routers se encargan de recibir solicitudes y retornar respuestas, evitando contener lógica de negocio compleja. Por otro lado, los schemas validan y estructuran los datos de entrada y salida de manera independiente. Finalmente, la capa de datos centraliza el acceso a persistencia, evitando que los endpoints interactúen directamente con la base de datos.

La estructura seleccionada facilita el mantenimiento y escalabilidad porque:

- reduce el acoplamiento entre componentes,
- permite modificar validaciones o persistencia sin afectar la API REST,
- facilita agregar nuevos dominios funcionales,
- permite escalar el proyecto agregando nuevos microservicios siguiendo la misma estructura,
- simplifica testing y debugging,
- mejora la mantenibilidad y evolución futura del backend.

3.5. 5. Alta cohesión de microservicios (10 pts)

Debe justificar: Por qué las clases pertenecen al mismo microservicio. Qué clases o funciones fueron descartadas o movidas. Cómo la decisión mejora la cohesión.

Decisión arquitectónica: División del sistema CESFAM en microservicios orientados a dominios específicos para garantizar alta cohesión.

Alternativa A: Agrupar entidades y funciones transaccionales de distintos contextos (pacientes, recetas médicas e inventario físico) en microservicios multipropósito o un componente monolítico.

Descarte: Se descarta porque mezcla responsabilidades puramente clínicas (estado de una receta) con responsabilidades logísticas (stock en bodega) y demográficas (filiación del paciente). Esto genera un bajo nivel de cohesión, haciendo que un mismo servicio deba ser modificado por reglas de negocio completamente distintas, aumentando la complejidad y el riesgo de errores al evolucionar el software (basta que falle uno solo para tirar todo el sistema).

Alternativa B: Definir fronteras estrictas separando el sistema en servicios dedicados (*Patient*, *Inventory*, *Prescription*, etc.) que encapsulan exclusivamente las operaciones de su propio dominio.

Selección: Alternativa B.

Justificación: Las funciones y entidades se agrupan porque comparten el mismo ciclo de vida, propósito unificado y reglas de validación. Por ejemplo, en el *InventoryService*, operaciones como *addBatch*, *writeOffBatch* y *consumePhysical* pertenecen juntas porque todas gestionan el estado logístico de la bodega. De igual forma, en *PatientService*, *updatePatient* y *addGuardian* coexisten porque la relación tutor-paciente es indivisible de la gestión de identidad de los usuarios de salud.

3.6. 6. Bajo acoplamiento de microservicios (15 pts)

Decisión arquitectónica: Usar un API Gateway siguiendo el patrón Backend-for-Frontend (BFF) para centralizar el acceso del frontend a los microservicios.

Alternativa A: Permitir que el frontend consuma directamente cada microservicio conociendo las URLs y endpoints internos de cada uno.

Descarte: Se descartó porque genera mucho acoplamiento entre el frontend y la estructura interna del backend. Si en el futuro cambia la URL de un servicio, se divide un microservicio o cambia la arquitectura interna, también habría que modificar el frontend. Se tendría que manejar una lógica de comunicación más compleja.

Alternativa B: Agregar un ApiGateway (BFF) que exponga endpoints más simples y preparados para el frontend, y que internamente se encargue de comunicarse con los demás servicios.

Selección: Se seleccionó la alternativa B.

Justificación: Esta opción reduce bastante el acoplamiento, porque el frontend solo conoce un contrato expuesto único expuesto por el Gateway. El cuál absorbe toda la complejidad de comunicación con los microservicios internos, permitiendo que los servicios puedan cambiar o evolucionar sin afectar directamente al cliente web. Ayuda a mantener mayor cohesión.

Decisión arquitectónica: Desacoplar la comunicación entre servicios usando clientes HTTP dedicados y mecanismos de resiliencia como Circuit Breaker.

Alternativa A: Permitir que un microservicio acceda directamente a los datos internos de otro servicio. Por ejemplo, que PrescriptionService lea directamente la estructura de datos o la tabla de InventoryService para verificar stock.

Descarte: Se descartó porque rompe la autonomía de los microservicios y genera acoplamiento de datos o estado compartido. Además, cualquier cambio interno en un servicio podría afectar inmediatamente a los demás, haciendo más difícil el mantenimiento y reduciendo la independencia de evolución de cada dominio.

Alternativa B: Hacer que toda comunicación entre dominios pase por clientes HTTP especializados (InventoryServiceClient, PatientServiceClient, etc.), usando contratos REST y respuestas estándar como ApiResponse<T>.

Selección: Se seleccionó la alternativa B.

Justificación: Al encapsular la comunicación en una capa de clientes dedicada, los servicios no necesitan conocer cómo están implementados internamente los demás microservicios, sino solo su API pública. Esto ayuda a mantener un bajo acoplamiento y permite que cada servicio pueda evolucionar o desplegarse de forma independiente mientras mantenga estable su contrato REST. Además, el uso de Circuit Breaker evita que una falla en un servicio (por ejemplo InventoryService) termine afectando completamente a otro como PrescriptionService, mejorando la resiliencia del sistema.

Decisión arquitectónica: Separar las estructuras de datos en memoria según el dominio de cada microservicio.

Alternativa A: Tener un único archivo global (por ejemplo un gran diccionario en main.py) con toda la información del sistema compartida entre todos los routers.

Descarte: Se descartó porque, aunque el backend sea un mock temporal en FastAPI, seguir usando un estado global compartido simula una arquitectura monolítica y genera alto acoplamiento entre dominios.

Alternativa B: Permitir que cada módulo o servicio mantenga sus propias estructuras de datos en memoria dentro de la carpeta data/, exponiendo información a otros servicios únicamente mediante contratos internos de API.

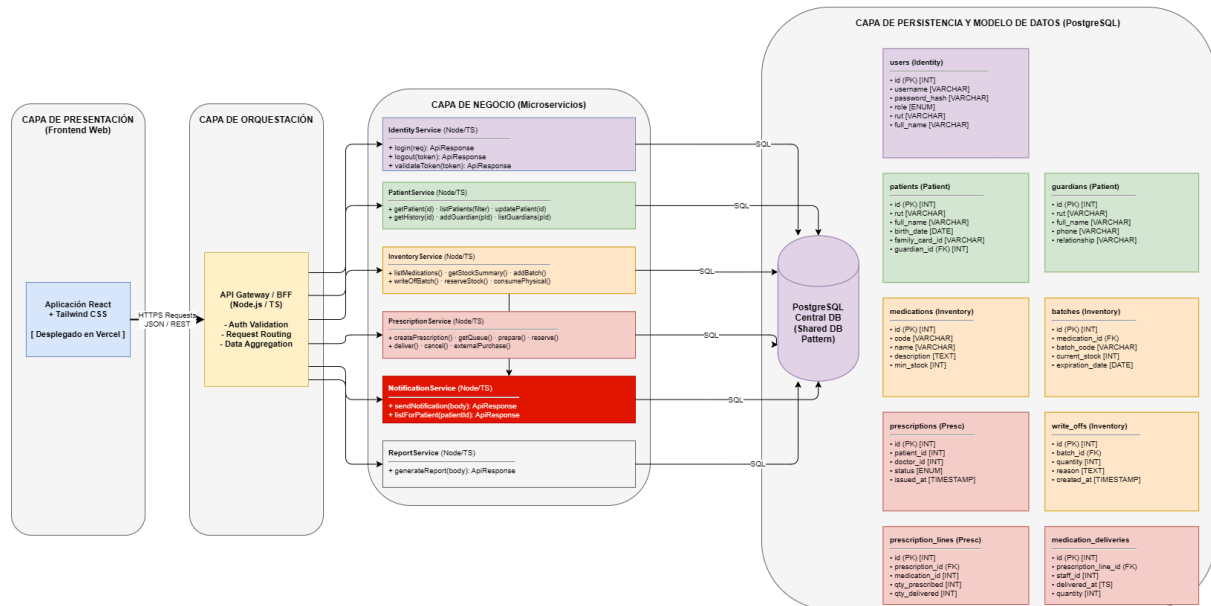
Selección: Se seleccionó la alternativa B.

Justificación: Esta opción permite mantener el bajo acoplamiento, cada microservicio controla únicamente sus propios datos y obliga a que la comunicación ocurra a través de interfaces públicas bien definidas.

4. Formato esperado para justificar decisiones

Diagrama de Arquitectura de Infraestructura y Datos.

Caso 17 — CESFAM: Arquitectura de Infraestructura y Datos (Entrega 2)



ANEXOS Y REFERENCIAS

Diagrama de Clases

Caso 17 — CESFAM
Diagrama de Clases

